

Tensorflow Guide



Meher Krishna Patel

Created on : September, 2018

Last updated : August, 2019

Table of contents

Table of contents	i
List of figures	iii
List of tables	iv
1 Tensorflow Overview	1
1.1 Introduction	1
1.2 Basic component of tensorflow	1
2 Tensorflow examples	4
2.1 Introduction	4
2.2 Convert to tensorflow	4
2.3 Operations	4
2.4 Save the tensorflow variables	5
2.5 Load the tensorflow variables	6
2.6 Implement $v = u + at$	6
2.7 Plot tensorflow graph	8
2.8 Questions	9
3 Simple Models	11
3.1 Introduction	11
3.2 Linear Regression model	11
3.2.1 Generate data	11
3.2.2 Linear regression model	11
3.3 Polynomial model	13
3.3.1 Apply polynomial model on linear data	14
3.3.2 Apply polynomial model on non linear data	15
4 Convolutional neural networks	18
4.1 Introduction	18
4.2 CIFAR dataset	18
4.2.1 Print labels in dataset	18
4.2.2 Plot data	18
4.3 Convolution filter	19
4.4 Convolve image with filter	20
4.4.1 Convolution layer	21
4.4.2 Add bias and Relu layer	22
4.4.3 Max pooling	24
4.5 Implement convolution network	27
5 MNIST example	29
5.1 Introduction	29
5.2 Read and Plot numbers from “mnist” dataset	29

5.3	Create tensorflow graph	30
5.4	Create model	32
5.5	Execute the model	32
5.6	Use Gradient optimizer for prediction	35

List of figures

2.1	Acceleration	9
2.2	Speed	10
2.3	Time	10
3.1	Scatter data : $y = 2 * x + 0.4 * \text{noise}$	12
3.2	Estimated values by the linear regression model	13
3.3	Polynomial model on linear data	15
3.4	Polynomial model on nonlinear data	17
4.1	First 9 images in the CIFAR dataset	20
4.2	Convolution filters	21
4.3	Output : image convolved with filter	23
4.4	Output after Relu	25
4.5	Output of maxpool layer	28
5.1	First nine numbers in test data	31
5.2	All the numbers are predicted as 0	36
5.3	Accuracy improved by using Gradient optimizer for prediction	39

List of tables

Chapter 1

Tensorflow Overview

1.1 Introduction

- Operates over Tensor (i.e. n-dim array)

1.2 Basic component of tensorflow

- **Graph** : The tensorflow computational steps are represented as a dataflow graph.
 - It is a collection of operations that may execute together as a group.
 - Since Tensorflow has the complete computational graph in the beginning of the calculation, therefore it is more efficient than the numpy which knows only one operation at a time.
 - Due to complete graph, Tensorflow can calculate the gradient which are required for optimized solutions.
 - Tensorflow can use both CPU, GPU and TPU (tensor processing unit).
- **Operations**: It is a graph node that performs computation on tensors.
- **Tensor**: It handles the one of the outputs of an Operations.
 - provide a means of computing a value in a Tensorflow session.
- **Placeholder variable**: It is used as input to the graph, which changes its value during graph-execution. In the other words, all the input values are passed to graph via placeholder.
- **Variables**: a modifiable tensor that lives in Tensorflow's graph of interacting operations. Values of these variables are changed by tensorflow, so that model can perform better on the training data.
- **Sessions**: encapsulate the environment in which operations objects are executed, and tensor objects are calculated.

```
1  # ex1.py
2
3  import numpy as np
4  import tensorflow as tf
5
6
7  # constant matrix from numpy
8  m1 = np.array([[1., 2.], [3., 4.], [5., 6.], [7., 8.]], dtype=np.float32)
9  # A placeholder to read the numpy input data.
10 m1_input = tf.placeholder(tf.float32, shape=[4, 2])
11
12 # variable using 'tensorflow'
13 m2 = tf.Variable(tf.random_uniform([2, 3], -1.0, 1.0))
14
15 # multiply matrix m1_input and m2
16 # note that we did not send m1 directly but the placeholder m1_input
17 # also, define below line does not multiply immediately
18 # it is just a graph which contains the list of operations.
```

(continues on next page)

(continued from previous page)

```

19 m3 = tf.matmul(m1_input, m2)
20
21
22 # Add variable initializer. It is required to initialize all the variables.
23 init = tf.global_variables_initializer()
24
25
26 # all the above commands will be run in the session.
27 with tf.Session() as session:
28     # We must initialize all variables before we use them.
29     init.run()
30     # print("Initialized")
31
32     print("m2: {}".format(m2)) # m2: <tf.Variable 'Variable:0' shape=(2, 3) dtype=float32_ref>
33     print("eval m2: {}".format(m2.eval())) # display the value in m2 (tensorflow-variable)
34
35     # map input value to placeholder, so that it can be used by tensorflow
36     map_inputs_dict = {m1_input: m1}
37
38     # calculate m3 based on values in map_inputs_dict
39     result = session.run([m3], feed_dict=map_inputs_dict)
40     print("\nresult: {}\n".format(result))

```

- Let's add the result m3 to itself,

```

1 # ex1.py
2
3 import numpy as np
4 import tensorflow as tf
5
6
7 # constant matrix from numpy
8 m1 = np.array([[1., 2.], [3., 4.], [5., 6.], [7., 8.]], dtype=np.float32)
9 # A placeholder to read the numpy input data.
10 m1_input = tf.placeholder(tf.float32, shape=[4, 2])
11
12 # variable using 'tensorflow'
13 m2 = tf.Variable(tf.random_uniform([2, 3], -1.0, 1.0))
14
15 # multiply matrix m1_input and m2
16 # note that we did not send m1 directly but the placeholder m1_input
17 # also, define below line does not multiply immediately
18 # it is just a graph which contains the list of operations.
19 m3 = tf.matmul(m1_input, m2)
20
21 # add result m3 to itself
22 m4 = tf.add(m3, m3)
23
24
25 # Add variable initializer. It is required to initialize all the variables.
26 init = tf.global_variables_initializer()
27
28
29 # all the above commands will be run in the session.
30 with tf.Session() as session:
31     # We must initialize all variables before we use them.
32     init.run()
33     # print("Initialized")
34
35     print("m2: {}".format(m2)) # m2: <tf.Variable 'Variable:0' shape=(2, 3) dtype=float32_ref>
36     print("eval m2: {}".format(m2.eval())) # display the value in m2 (tensorflow-variable)

```

(continues on next page)

(continued from previous page)

```
37
38     # map input value to placeholder, so that it can be used by tensorflow
39     map_inputs_dict = {m1_input: m1}
40
41     # calculate m3 based on values in map_inputs_dict
42     mul1, sum1 = session.run([m3, m4], feed_dict=map_inputs_dict)
43     print("m3: {}".format(mul1))
44     print("m4: {}".format(sum1))
```

Chapter 2

Tensorflow examples

2.1 Introduction

In [Chapter 1](#), we saw an overview to Tensorflow. In this chapter, we will see various operations which are available in the tensorflow. From the next chapter, we will use these operations to implement the machine learning algorithm.

2.2 Convert to tensorflow

```
1 >>> import tensorflow as tf
2
3 # list to tensorflow conversion
4 >>> m1 = [ [1, 2], [3, 4] ]
5 >>> type(m1)
6 <class 'list'>
7 >>> t1 = tf.convert_to_tensor(m1, dtype=tf.float32)
8 >>> type(t1)
9 <class 'tensorflow.python.framework.ops.Tensor'>
10
11
12 # numpy to tensorflow conversion
13 >>> import numpy as np
14 >>> m2 = np.array([[1, 2], [3, 4]], dtype=np.float32)
15 >>> type(m2)
16 <class 'numpy.ndarray'>
17 >>> t2 = tf.convert_to_tensor(m2, dtype=tf.float32)
18 >>> type(t2)
19 <class 'tensorflow.python.framework.ops.Tensor'>
```

2.3 Operations

In previous section, we saw the methods which is used convert the list or numpy array into tensors. Now, we will perform some operations on the tensorflow. The complete list of operations can be found at [tf.math](#).

```
1 >>> import tensorflow as tf
2
3 # calculate -ve
4 >>> a = tf.constant([ [1, 2, 3, 4] ])
5 >>> b = tf.negative(a)
6 >>> print(b)
```

(continues on next page)

(continued from previous page)

```

7 Tensor("Neg:0", shape=(1, 4), dtype=int32)
8 >>>
9 >>> with tf.Session() as sess:
10 ...     result = sess.run(b)
11 ...     print(result)
12 ...
13 [[-1 -2 -3 -4]]
14
15 # add two tensors
16 >>> sum = tf.add(a, b);
17 >>> with tf.Session() as sess:
18 ...     sum = sess.run(sum)
19 ...     print(sum)
20 ...
21 [[0 0 0 0]]

```

2.4 Save the tensorflow variables

```

1 # save_tf.py
2
3 import tensorflow as tf
4
5 a = tf.constant([ [1, 2, 3, 4] ], name="a")
6 b = tf.Variable(tf.random_normal(shape=[2]), name="b")
7 c = tf.Variable(tf.random_normal(shape=[2,2]), name="c")
8
9 init = tf.global_variables_initializer()
10 saver = tf.train.Saver()
11
12 with tf.Session() as sess:
13     init.run()
14     d = tf.multiply(a, 2)
15     e = tf.multiply(c, 3)
16     saver_path = saver.save(sess, "./meher.ckpt")
17
18     print("\n\n")
19     print("a = ", a.eval())
20     print("b = ", b.eval())
21     print("c = ", c.eval())
22     print("d = ", d.eval())
23     print("\n\nSaver Path = ", saver_path)
24
25 print("\n\nSaved items in the checkpoint are below:\n")
26 # see all values which are stored in the checkpoint
27 from tensorflow.python.tools.inspect_checkpoint import print_tensors_in_checkpoint_file
28 latest_ckp = tf.train.latest_checkpoint('./')
29 print_tensors_in_checkpoint_file(latest_ckp, all_tensors=True, tensor_name='')

```

- Below is the output,

```

a =  [[1 2 3 4]]
b =  [0.5460819 0.6374034]
c =  [[ 0.09093301 -0.3235254 ]
 [-0.18190224  0.01609637]]
d =  [[2 4 6 8]]

Saver Path =  ./meher.ckpt

```

(continues on next page)

(continued from previous page)

Saved items in the checkpoint are below:

```

tensor_name:  b
[0.5460819  0.6374034]

tensor_name:  c
[[ 0.09093301 -0.3235254 ]
 [-0.18190224  0.01609637]]

```

2.5 Load the tensorflow variables

```

# load_tf.py

import tensorflow as tf

with tf.Session() as sess:
    saver = tf.train.import_meta_graph('meher.ckpt.meta')
    saver.restore(sess,tf.train.latest_checkpoint('./'))

    graph = tf.get_default_graph()
    x = graph.get_tensor_by_name("b:0")
    y = graph.get_tensor_by_name("c:0")

    print("x = ", x.eval())
    print("y = ", y.eval())

```

- Below is the output,

```

$ python load_tf.py

x =  [0.5460819  0.6374034]
y =  [[ 0.09093301 -0.3235254 ]
 [-0.18190224  0.01609637]]

```

2.6 Implement $v = u + at$

```

1 # eq_test.py
2
3 import tensorflow as tf
4 import numpy as np
5
6 sample_data = np.arange(1, 10)
7
8 time = tf.placeholder(tf.float32, name="time")
9 initial_speed = tf.constant(10., name="initial_speed")
10 acc = tf.Variable(2., name="acc")
11
12
13 final_speed = initial_speed + acc * time
14 init = tf.global_variables_initializer()
15
16 saver = tf.train.Saver()
17

```

(continues on next page)

(continued from previous page)

```

18 with tf.Session() as sess:
19     sess.run(init)
20
21     print("\n\n")
22     print("Initial acceleration is 2")
23     print("Final speed = 10 + 2 * t")
24     for i in range(len(sample_data)):
25         inst_speed = sess.run(final_speed, feed_dict = {time : sample_data[i]})
26         print("Time: {}, Final speed: {}".format(sample_data[i], inst_speed))
27
28
29     print("\n\n")
30     print("Changing acceleration to 4")
31     print("Final speed = 10 + 4 * t")
32     sess.run(tf.assign(acc, 4.0))
33     for i in range(len(sample_data)):
34         inst_speed = sess.run(final_speed, feed_dict = {time : sample_data[i]})
35         print("Time: {}, Final speed: {}".format(sample_data[i], inst_speed))
36
37     saver_path = saver.save(sess, "./eq_test.ckpt")
38     print("\n\nSaver Path = ", saver_path)
39
40 print("\n\nSaved items in the checkpoint are below:\n")
41 # see all values which are stored in the checkpoint
42 from tensorflow.python.tools.inspect_checkpoint import print_tensors_in_checkpoint_file
43 latest_ckp = tf.train.latest_checkpoint('.')
44 print_tensors_in_checkpoint_file(latest_ckp, all_tensors=True, tensor_name='')

```

- Below is the output,

```

$ python eq_test.py

Initial acceleration is 2
Final speed = 10 + 2 * t
Time: 1, Final speed: 12.0
Time: 2, Final speed: 14.0
Time: 3, Final speed: 16.0
Time: 4, Final speed: 18.0
Time: 5, Final speed: 20.0
Time: 6, Final speed: 22.0
Time: 7, Final speed: 24.0
Time: 8, Final speed: 26.0
Time: 9, Final speed: 28.0

Changing acceleration to 4
Final speed = 10 + 4 * t
Time: 1, Final speed: 14.0
Time: 2, Final speed: 18.0
Time: 3, Final speed: 22.0
Time: 4, Final speed: 26.0
Time: 5, Final speed: 30.0
Time: 6, Final speed: 34.0
Time: 7, Final speed: 38.0
Time: 8, Final speed: 42.0
Time: 9, Final speed: 46.0

Saver Path = ./eq_test.ckpt

```

(continues on next page)

(continued from previous page)

Saved items in the checkpoint are below:

```
tensor_name:  acc
4.0
```

Note: The updated value of the acc (i.e. 4) is stored in the checkpoint (initial was 2).

2.7 Plot tesnorflow graph

```

1  # eq_test.py
2
3  import tensorflow as tf
4  import numpy as np
5
6  sample_data = np.arange(1, 10)
7
8  time = tf.placeholder(tf.float32, name="time")
9  initial_speed = tf.constant(10., name="initial_speed")
10 acc = tf.Variable(2., name="acc")
11
12
13 final_speed = initial_speed + acc * time
14 init = tf.global_variables_initializer()
15
16 writer = tf.summary.FileWriter("./log/")
17 saver = tf.train.Saver()
18
19 with tf.Session() as sess:
20     sess.run(init)
21
22     # add 3 summary, therefore 3 graph will be generated
23     tf.summary.scalar('acc', acc)
24     tf.summary.scalar('time', time)
25     tf.summary.scalar('final_speed', final_speed)
26     merged = tf.summary.merge_all() # merge above graph
27
28     print("\n\n")
29     print("Initial acceleration is 2")
30     print("Final speed = 10 + 2 * t")
31     for i in range(int(len(sample_data)/2)):
32         summary, inst_speed = sess.run([merged, final_speed], feed_dict = {time : sample_data[i]})
33         print("Time: {}, Final speed: {}".format(sample_data[i], inst_speed))
34         writer.add_summary(summary, i)
35
36
37     print("\n\n")
38     print("Changing acceleration to 4")
39     print("Final speed = 10 + 4 * t")
40     sess.run(tf.assign(acc, 4.0))
41     for i in range(int(len(sample_data)/2), len(sample_data)):
42         summary, inst_speed = sess.run([merged, final_speed], feed_dict = {time : sample_data[i]})
43         print("Time: {}, Final speed: {}".format(sample_data[i], inst_speed))
44         writer.add_summary(summary, i)
45
46     saver_path = saver.save(sess, "./eq_test.ckpt")

```

(continues on next page)

(continued from previous page)

```

47     print("\nSaver Path = ", saver_path)
48
49
50 print("\nSaved items in the checkpoint are below:\n")
51 # see all values which are stored in the checkpoint
52 from tensorflow.python.tools.inspect_checkpoint import print_tensors_in_checkpoint_file
53 latest_ckp = tf.train.latest_checkpoint('./')
54 print_tensors_in_checkpoint_file(latest_ckp, all_tensors=True, tensor_name='')

```

See the graph as below,

```

# remove the log folder
$ rm -r log/

# run the eq_test.py file
$ python eq_test.py

# see the graph
$ tensorboard --logdir=./log/
TensorBoard 1.9.0 at http://mekrip:6006 (Press CTRL+C to quit)

```

- Below are the graph for acceleration, speed and time

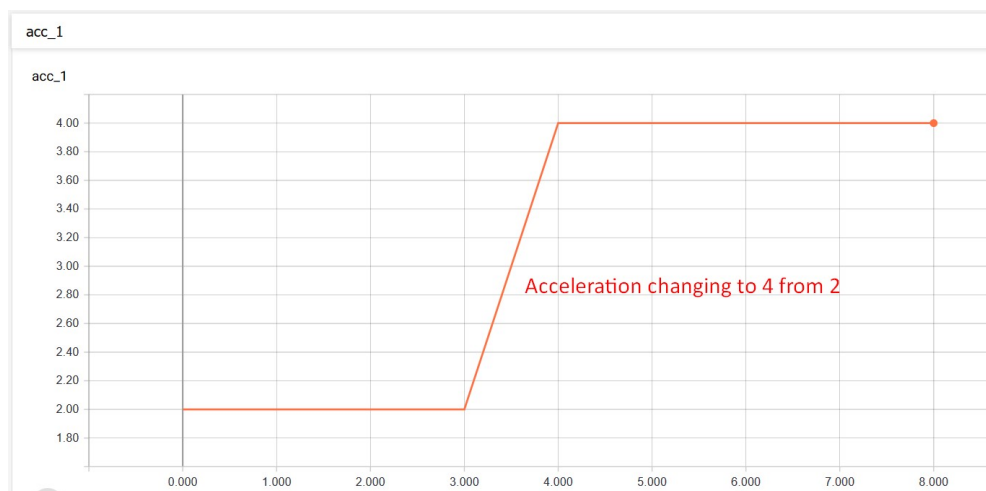


Fig. 2.1: Acceleration

2.8 Questions

1. Generate Gaussian distribution

```

1 from math import pi
2 mean = 0.0
3 sigma = 1.0
4 (tf.exp(tf.negative(tf.pow(x - mean, 2.0) /
5 (2.0 * tf.pow(sigma, 2.0) ))) *
6 (1.0 / (sigma * tf.sqrt(2.0 * pi) )))

```

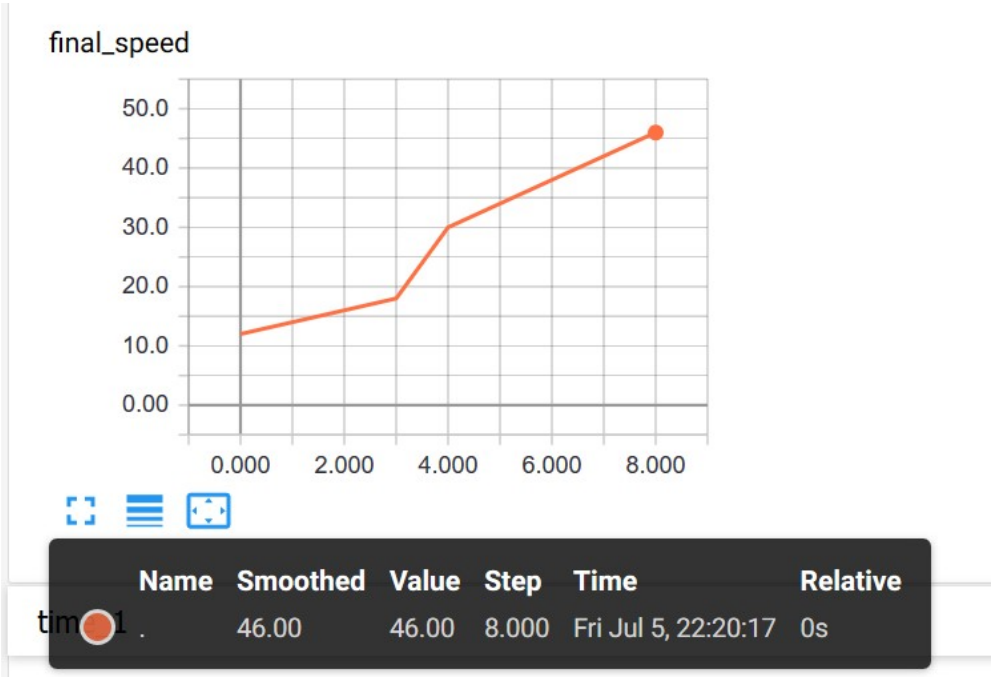


Fig. 2.2: Speed

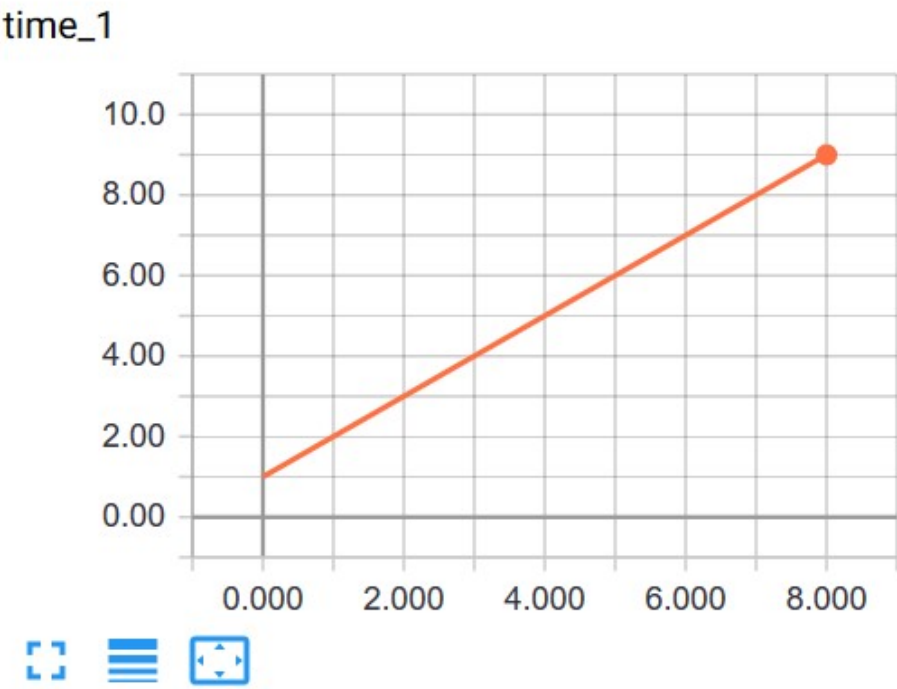


Fig. 2.3: Time

Chapter 3

Simple Models

3.1 Introduction

In this chapter, we will implement some of the models using tensorflow.

3.2 Linear Regression model

The equation for linear regression model is given by $y = wx$

3.2.1 Generate data

```
1 # linear_reg.py
2
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import tensorflow as tf
6
7 def generate_data():
8     x_train = np.linspace(-1, 1, 100)
9
10    # *x_train.shape results the value of tuple i.e. 3 for (3, )
11    y_train = 0.4 * np.random.randn(*x_train.shape) + 2 * x_train
12
13    return x_train, y_train
14
15 if __name__ == '__main__':
16     x_train, y_train = generate_data()
17     plt.scatter(x_train, y_train)
18     plt.show()
```

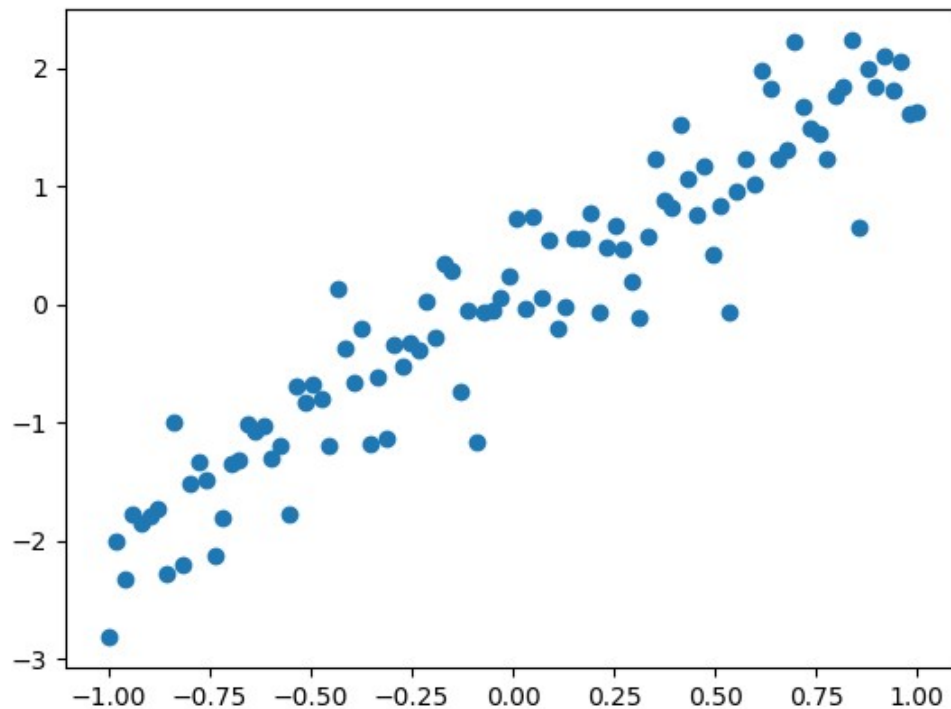
- Below is the output,

```
$ python linear_reg.py
```

3.2.2 Linear regression model

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import tensorflow as tf
```

(continues on next page)

Fig. 3.1: Scatter data : $y = 2 * x + 0.4 * \text{noise}$

(continued from previous page)

```

4
5 def generate_data():
6     x_train = np.linspace(-1, 1, 100)
7
8     # *x_train.shape results the value of tuple i.e. 3 for (3, )
9     y_train = 0.4 * np.random.randn(*x_train.shape) + 2 * x_train
10
11     return x_train, y_train
12
13 def linear_reg_model(X, w):
14
15     return tf.multiply(X, w)
16
17 def run_session(x_train, y_train, learning_rate, epochs):
18     X = tf.placeholder(tf.float32) # training input
19     Y = tf.placeholder(tf.float32) # expected output
20
21     # weights
22     w = tf.Variable(0.0, name="weights")
23
24     # linear regression model
25     y_model = linear_reg_model(X, w)
26
27     # cost
28     cost = tf.square(Y - y_model)
29
30     # optimizer
31     train_op = tf.train.GradientDescentOptimizer(learning_rate).minimize(cost)
32
33     # start session
34     sess = tf.Session()

```

(continues on next page)

(continued from previous page)

```

35 init = tf.global_variables_initializer()
36 sess.run(init)
37
38 for epoch in range(epochs):
39     for (x, y) in zip(x_train, y_train):
40         sess.run(train_op, feed_dict={X: x, Y: y})
41         w_val = sess.run(w)
42
43 sess.close()
44
45 return w_val
46
47 if __name__ == '__main__':
48     x_train, y_train = generate_data()
49     plt.scatter(x_train, y_train)
50
51     w_val = run_session(x_train, y_train, learning_rate=0.01, epochs=100)
52     y_estimated = x_train * w_val
53     plt.plot(x_train, y_estimated, '-r')
54
55 plt.show()

```

- Below is the output,

```
$ python linear_reg.py
```

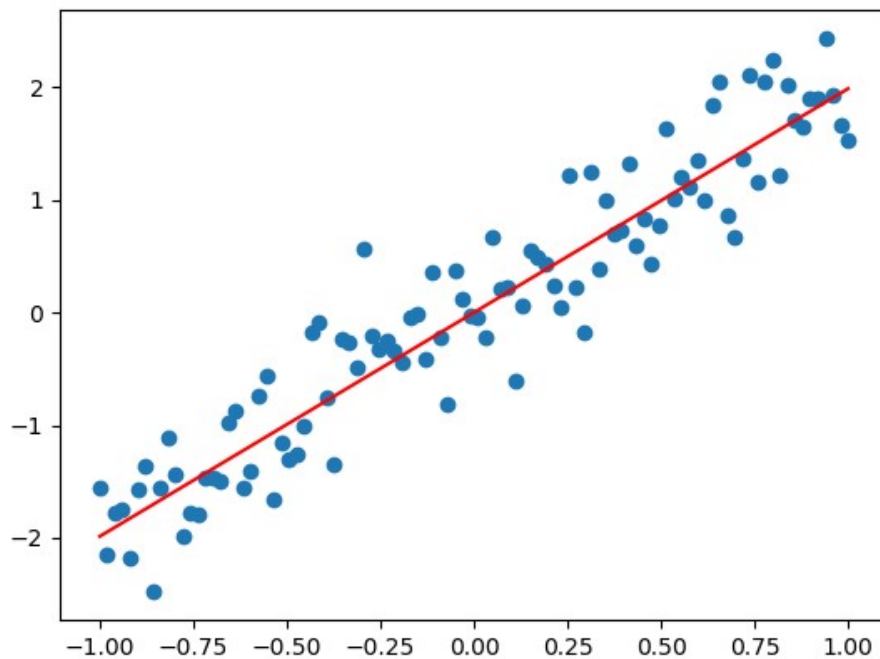


Fig. 3.2: Estimated values by the linear regression model

3.3 Polynomial model

The equation for polynomial model is given by $y = w_n x^n + \dots + w_1 x + w_0$

3.3.1 Apply polynomial model on linear data

In this section, we used the polynomial model on the linear data which is generated in last section,

```

1  # polynomial_ex.py
2
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import tensorflow as tf
6
7  def generate_data():
8      x_train = np.linspace(-1, 1, 100)
9
10     # *x_train.shape results the value of tuple i.e. 3 for (3, )
11     y_train = 0.4 * np.random.randn(*x_train.shape) + 2 * x_train
12
13     return x_train, y_train
14
15  def polynomial_model(X, w, num_coeff):
16      terms = []
17      for i in range(num_coeff):
18          terms.append(tf.multiply(w[i], tf.pow(X, i)))
19      return tf.add_n(terms)
20
21  def run_session(x_train, y_train, num_coeff, learning_rate, epochs):
22      X = tf.placeholder(tf.float32) # training input
23      Y = tf.placeholder(tf.float32) # expected output
24
25      # weights
26      w = tf.Variable([0.] * num_coeff, name="weights")
27
28      # linear regression model
29      y_model = polynomial_model(X, w, num_coeff=num_coeff)
30
31      # cost
32      cost = tf.square(Y - y_model)
33
34      # optimizer
35      train_op = tf.train.GradientDescentOptimizer(learning_rate).minimize(cost)
36
37      # start session
38      sess = tf.Session()
39      init = tf.global_variables_initializer()
40      sess.run(init)
41
42      for epoch in range(epochs):
43          for (x, y) in zip(x_train, y_train):
44              sess.run(train_op, feed_dict={X: x, Y: y})
45              w_val = sess.run(w)
46
47      sess.close()
48
49      return w_val
50
51  if __name__ == '__main__':
52      x_train, y_train = generate_data()
53      plt.scatter(x_train, y_train)
54
55      num_coeff = 6
56      w_val = run_session(x_train, y_train, num_coeff=num_coeff, learning_rate=0.01, epochs=100)
57
58      y_estimated = 0

```

(continues on next page)

(continued from previous page)

```

59     for i in range(num_coeff):
60         y_estimated += w_val[i] * np.power(x_train, i)
61
62     plt.plot(x_train, y_estimated, '-r')
63
64     plt.show()

```

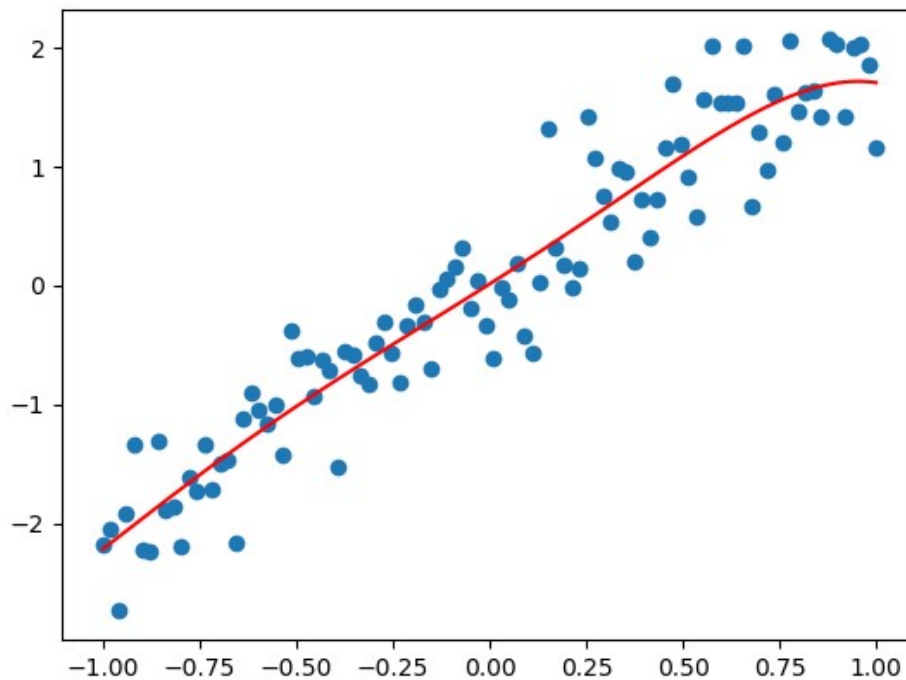


Fig. 3.3: Polynomial model on linear data

3.3.2 Apply polynomial model on non linear data

Now, generate the nonlinear data in function “generate_data” and see the performance of the code.

```

1  # polynomial_ex.py
2
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import tensorflow as tf
6
7  def generate_data():
8      x_train = np.linspace(-1, 1, 100)
9
10     # *x_train.shape results the value of tuple i.e. 3 for (3, )
11     y_train = 0
12     for i in range(10):
13         y_train += 0.4 * np.random.randn(*x_train.shape) + 2 * np.power(x_train, i)
14
15     return x_train, y_train
16
17 def polynomial_model(X, w, num_coeff):
18     terms = []
19     for i in range(num_coeff):
20         terms.append(tf.multiply(w[i], tf.pow(X, i)))

```

(continues on next page)

(continued from previous page)

```

21     return tf.add_n(terms)
22
23 def run_session(x_train, y_train, num_coeff, learning_rate, epochs):
24     X = tf.placeholder(tf.float32) # training input
25     Y = tf.placeholder(tf.float32) # expected output
26
27     # weights
28     w = tf.Variable([0.] * num_coeff, name="weights")
29
30     # linear regression model
31     y_model = polynomial_model(X, w, num_coeff=num_coeff)
32
33     # cost
34     cost = tf.square(Y - y_model)
35
36     # optimizer
37     train_op = tf.train.GradientDescentOptimizer(learning_rate).minimize(cost)
38
39     # start session
40     sess = tf.Session()
41     init = tf.global_variables_initializer()
42     sess.run(init)
43
44     for epoch in range(epochs):
45         for (x, y) in zip(x_train, y_train):
46             sess.run(train_op, feed_dict={X: x, Y: y})
47             w_val = sess.run(w)
48
49     sess.close()
50
51     return w_val
52
53 if __name__ == '__main__':
54     x_train, y_train = generate_data()
55     plt.scatter(x_train, y_train)
56
57     num_coeff = 6
58     w_val = run_session(x_train, y_train, num_coeff=num_coeff, learning_rate=0.01, epochs=100)
59
60     y_estimated = 0
61     for i in range(num_coeff):
62         y_estimated += w_val[i] * np.power(x_train, i)
63
64     plt.plot(x_train, y_estimated, '-r')
65
66     plt.show()

```

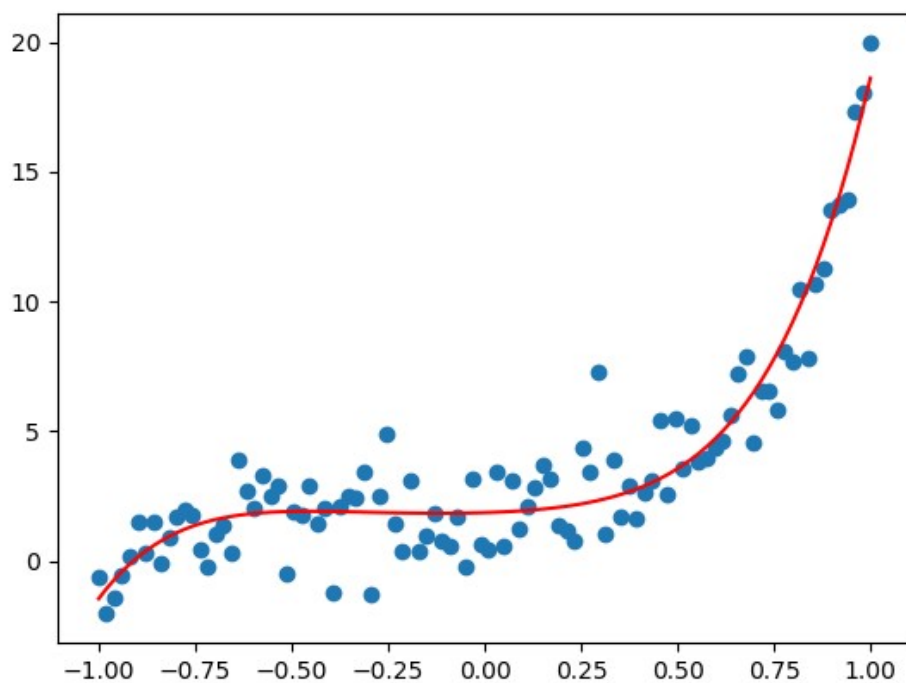


Fig. 3.4: Polynomial model on nonlinear data

Chapter 4

Convolutional neural networks

4.1 Introduction

In this chapter, we will use the CIFAR dataset to understand the Convolutional Neural Network (CNN). Please download the CIFAR dataset for Python from <http://www.cs.toronto.edu/~kriz/cifar.html>.

4.2 CIFAR dataset

4.2.1 Print labels in dataset

```
1 # cifar_dataset.py
2
3 import pickle
4
5 def unpickle(file):
6     fo = open(file, 'rb')
7     dict = pickle.load(fo, encoding='latin1')
8     fo.close()
9     return dict
10
11 if __name__=="__main__":
12     directory = 'C:/meher/Git/cifar-10-batches-py'
13     names = unpickle('{}/batches.meta'.format(directory))
14     print(names['label_names']) # print class names
```

- Below is the output

```
$ python cifar_dataset.py
['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```

4.2.2 Plot data

In the below result, the data at location 0 is 'frog'; i.e. label = 6 which is 'frog' in the variable 'names'.

```
1 # cifar_dataset.py
2
3 import pickle
4 import numpy as np
5 import matplotlib.pyplot as plt
6
```

(continues on next page)

(continued from previous page)

```

7 def unpickle(file):
8     fo = open(file, 'rb')
9     dict = pickle.load(fo, encoding='latin1')
10    fo.close()
11    return dict
12
13 def read_data(directory):
14     # get classes
15     names = unpickle('{} /batches.meta'.format(directory))['label_names']
16
17     # get data and labels
18     data, labels = [], []
19     for i in range(1, 6): # there are 5 files: data_batch_1 to data_batch_5
20         filename = '{} /data_batch_{}'.format(directory, i)
21         batch_data = unpickle(filename)
22         # dict_keys(['batch_label', 'labels', 'data', 'filenames'])
23         # print(batch_data.keys())
24         if len(data) == 0:
25             data = batch_data['data'] # extract data from the dict
26             labels = batch_data['labels'] # extract key from the dict
27         else:
28             # append data from next dict
29             data = np.vstack((data, batch_data['data']))
30             # append labels from next dict
31             labels = np.hstack((labels, batch_data['labels']))
32     return names, data, labels
33
34 if __name__ == "__main__":
35     directory = 'C:/meher/Git/cifar-10-batches-py'
36     # names = unpickle('{} /batches.meta'.format(directory))
37     # print(names['label_names']) # print class names
38     names, data, labels = read_data(directory)
39
40     item = 5
41     print("Names: ", names)
42     print("data: ", data[item])
43     print("item: {}, labels: {}, name: {}".format(item,
44                                                    labels[item], names[labels[item]]))
45
46     rows, cols = 3, 3 # show 9 images
47     for item in range(0, rows*cols):
48         img = np.reshape(data[item], (3, 32, 32)).transpose(1,2,0)
49         # img = data[item].reshape((3, 32, 32)).transpose(1,2,0)
50         plt.subplot(rows, cols, item+1)
51         plt.imshow(img)
52         plt.title(names[labels[item]])
53
54     plt.tight_layout()
55     plt.show()

```

- Below is result of above code,

4.3 Convolution filter

In the below code, 32 Convolution filters are generated and plotted.

```

1 # filter_example.py
2
3 import numpy as np

```

(continues on next page)

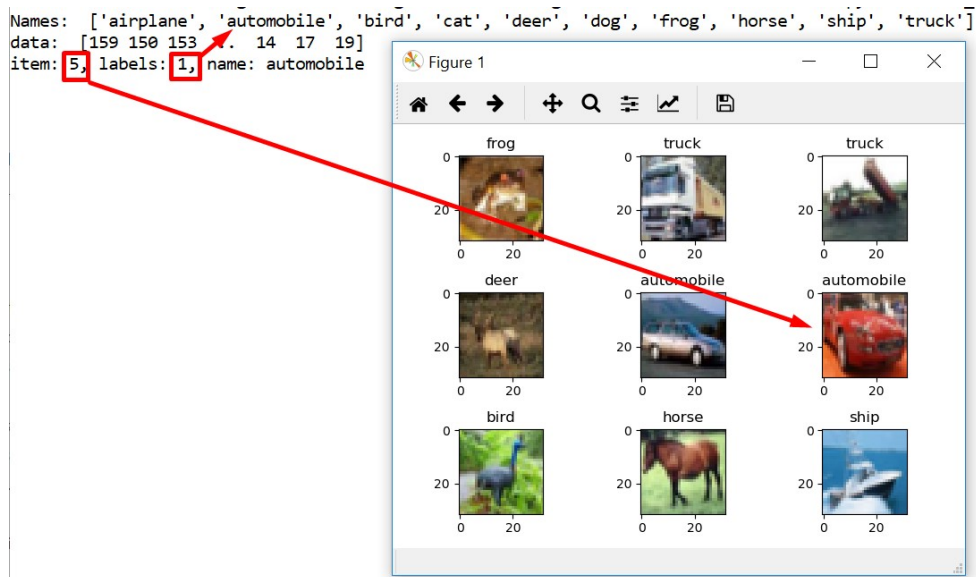


Fig. 4.1: First 9 images in the CIFAR dataset

(continued from previous page)

```

4 import matplotlib.pyplot as plt
5 import tensorflow as tf
6
7
8 def plot_filter(conv_filt):
9     plt.figure()
10    rows, cols = 4, 8
11    for i in range(np.shape(conv_filt)[3]):
12        img = conv_filt[:, :, 0, i] # i.e 5:5:1:i
13        plt.subplot(rows, cols, i + 1) # plot individual filter in subplot
14        plt.imshow(img)
15        plt.axis('off')
16        plt.title('Filter {}'.format(i))
17
18
19
20 if __name__=="__main__":
21     # generate 32 one dimensional filter of size 5, 5
22     conv_filt = tf.Variable(tf.random_normal([5, 5, 1, 32]))
23
24     with tf.Session() as sess:
25         sess.run(tf.global_variables_initializer())
26         conv_filt_val = sess.run(conv_filt)
27         plot_filter(conv_filt_val)
28
29 plt.show()

```

- Below are the 32 convolution filters which are generated by above code,

4.4 Convolve image with filter

In the below code, 16 filter of size 3x3 are used to convolve with the image to show the function of convolution filter; next we added the Relu layer to add the nonlinearity in the system. Later, we will create as separate function to define the model.

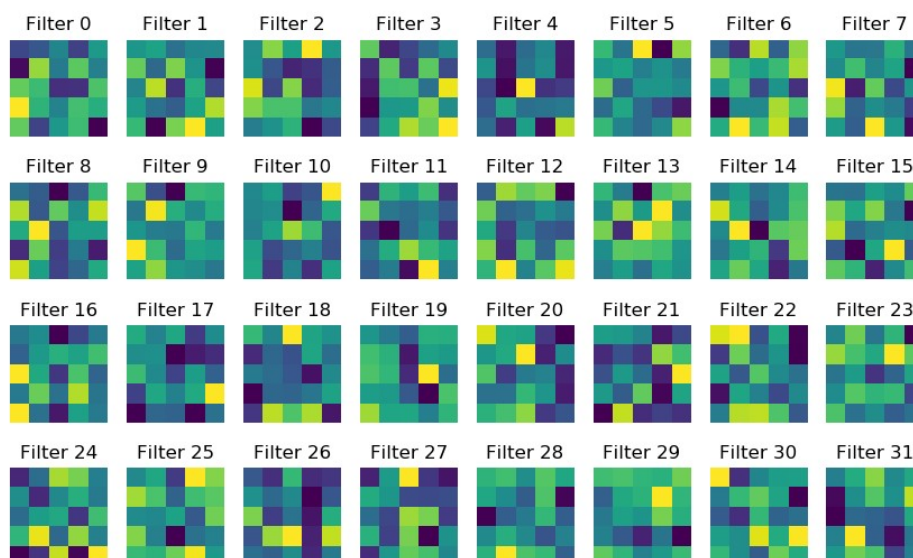


Fig. 4.2: Convolution filters

4.4.1 Convolution layer

```
# convolve_data.py

import pickle
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

def unpickle(file):
    fo = open(file, 'rb')
    dict = pickle.load(fo, encoding='latin1')
    fo.close()
    return dict

def read_data(directory):
    # get classes
    names = unpickle('{} /batches.meta'.format(directory))['label_names']

    # get data and labels
    data, labels = [], []
    for i in range(1, 6): # there are 5 files: data_batch_1 to data_batch_5
        filename = '{} /data_batch_{}'.format(directory, i)
        batch_data = unpickle(filename)
        # dict_keys(['batch_label', 'labels', 'data', 'filenames'])
        # print(batch_data.keys())
        if len(data) == 0:
            data = batch_data['data'] # extract data from the dict
            labels = batch_data['labels'] # extract key from the dict
        else:
            # append data from next dict
            data = np.vstack((data, batch_data['data']))
            # append labels from next dict
            labels = np.hstack((labels, batch_data['labels']))
    return names, data, labels
```

(continues on next page)

(continued from previous page)

```

def show_conv_results(data, rows=4, cols=4):
    plt.figure()
    rows, cols = rows, cols
    for i in range(np.shape(data)[3]):
        img = data[0, :, :, i]
        plt.subplot(rows, cols, i + 1)
        plt.imshow(img)
        plt.title("Filter {}".format(i))
        plt.axis('off')

if __name__=="__main__":
    directory = 'C:/meher/Git/cifar-10-batches-py'
    # names = unpickle('{} /batches.meta'.format(directory))
    # print(names['label_names']) # print class names
    names, data, labels = read_data(directory)

    item = 5
    # print("Names: ", names)
    # print("data: ", data[item])
    # print("item: {}, labels: {}, name: {}".format(item,
    #                                             labels[item], names[labels[item]]))

    # rows, cols = 3, 3 # show 9 images
    # for item in range(0, rows*cols):
    #     img = np.reshape(data[item], (3, 32, 32)).transpose(1,2,0)
    #     # img = data[item].reshape((3, 32, 32)).transpose(1,2,0)
    #     plt.subplot(rows, cols, item+1)
    #     plt.imshow(img)
    #     plt.title(names[labels[item]])

    img = data[item, :]
    img = tf.cast(img, tf.float32)
    img = tf.reshape(img, shape=[-1, 32, 32, 1])

    total_conv_filt = 16
    conv_filt_size = 3
    conv_filt = tf.Variable(tf.random_normal([conv_filt_size, conv_filt_size, 1, total_conv_filt]))
    conv = tf.nn.conv2d(img, conv_filt, strides=[1, 1, 1, 1], padding='SAME')

    with tf.Session() as sess:
        sess.run(tf.global_variables_initializer())
        conv_val = sess.run(conv)
        show_conv_results(conv_val, rows=4, cols=4)
        print(np.shape(conv_val))

    # plt.tight_layout()
    plt.show()

```

- Below are the corresponding 16 outputs of 3x3 convolution filters

4.4.2 Add bias and Relu layer

- In the below code, bias and relu layer are added in the design. And the output is plotted after relu layer.

```

# convolve_data.py

import pickle

```

(continues on next page)

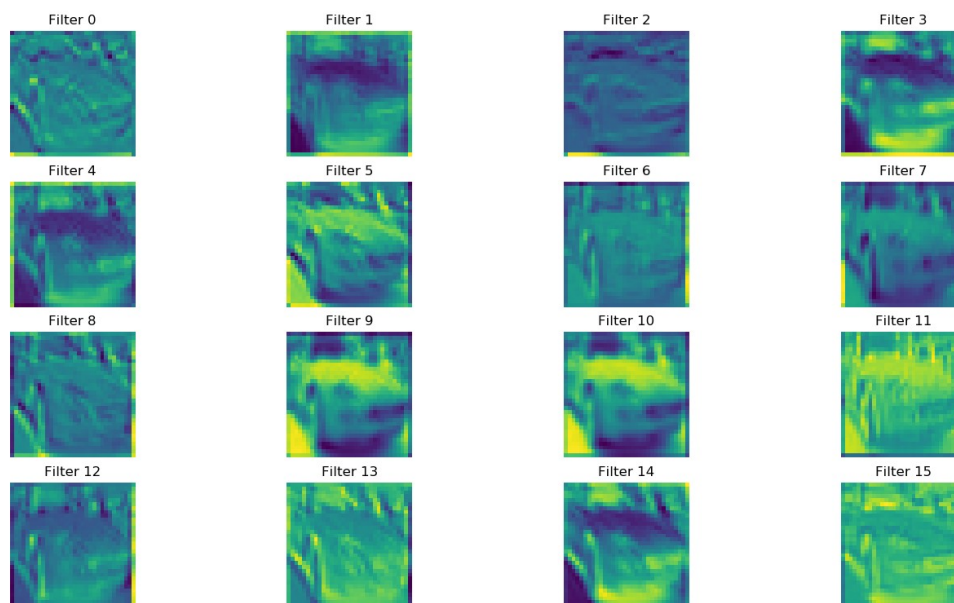


Fig. 4.3: Output : image convolved with filter

(continued from previous page)

```

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

def unpickle(file):
    fo = open(file, 'rb')
    dict = pickle.load(fo, encoding='latin1')
    fo.close()
    return dict

def read_data(directory):
    # get classes
    names = unpickle('{}batches.meta'.format(directory))['label_names']

    # get data and labels
    data, labels = [], []
    for i in range(1, 6): # there are 5 files: data_batch_1 to data_batch_5
        filename = '{}data_batch_{}'.format(directory, i)
        batch_data = unpickle(filename)
        # dict_keys(['batch_label', 'labels', 'data', 'filenames'])
        # print(batch_data.keys())
        if len(data) == 0:
            data = batch_data['data'] # extract data from the dict
            labels = batch_data['labels'] # extract key from the dict
        else:
            # append data from next dict
            data = np.vstack((data, batch_data['data']))
            # append labels from next dict
            labels = np.hstack((labels, batch_data['labels']))
    return names, data, labels

def show_conv_results(data, rows=4, cols=4):
    plt.figure()
    rows, cols = rows, cols
    for i in range(np.shape(data)[3]):

```

(continues on next page)

(continued from previous page)

```

        img = data[0, :, :, i]
        plt.subplot(rows, cols, i + 1)
        plt.imshow(img)
        plt.title("Filter {}".format(i))
        plt.axis('off')

if __name__=="__main__":
    directory = 'C:/meher/Git/cifar-10-batches-py'
    # names = unpickle('{}/batches.meta'.format(directory))
    # print(names['label_names']) # print class names
    names, data, labels = read_data(directory)

    item = 5
    # print("Names: ", names)
    # print("data: ", data[item])
    # print("item: {}, labels: {}, name: {}".format(item,
    #                                               labels[item], names[labels[item]]))

    # rows, cols = 3, 3 # show 9 images
    # for item in range(0, rows*cols):
    #     img = np.reshape(data[item], (3, 32, 32)).transpose(1,2,0)
    #     # img = data[item].reshape((3, 32, 32)).transpose(1,2,0)
    #     plt.subplot(rows, cols, item+1)
    #     plt.imshow(img)
    #     plt.title(names[labels[item]])

    img = data[item, :]
    img = tf.cast(img, tf.float32)
    img = tf.reshape(img, shape=[-1, 32, 32, 1])

    total_conv_filt = 16
    conv_filt_size = 3
    b = tf.Variable(tf.random_normal([total_conv_filt]))
    conv_filt = tf.Variable(tf.random_normal([conv_filt_size, conv_filt_size, 1, total_conv_filt]))
    conv_with_b = tf.nn.bias_add(conv_filt, b)
    conv_out = tf.nn.relu(conv_with_b)
    conv = tf.nn.conv2d(img, conv_out, strides=[1, 1, 1, 1], padding='SAME')

    with tf.Session() as sess:
        sess.run(tf.global_variables_initializer())
        conv_val = sess.run(conv)
        show_conv_results(conv_val, rows=4, cols=4)
        print(np.shape(conv_val))

    # plt.tight_layout()
    plt.show()

```

- Below are the corresponding 16 outputs of 3x3 convolution network after relu layer,

4.4.3 Max pooling

In the below code, 3x3 max pooling is applied on the output of Relu layer. This layer can be seen as regularization step.

```

1 # convolve_data.py
2
3 import pickle

```

(continues on next page)

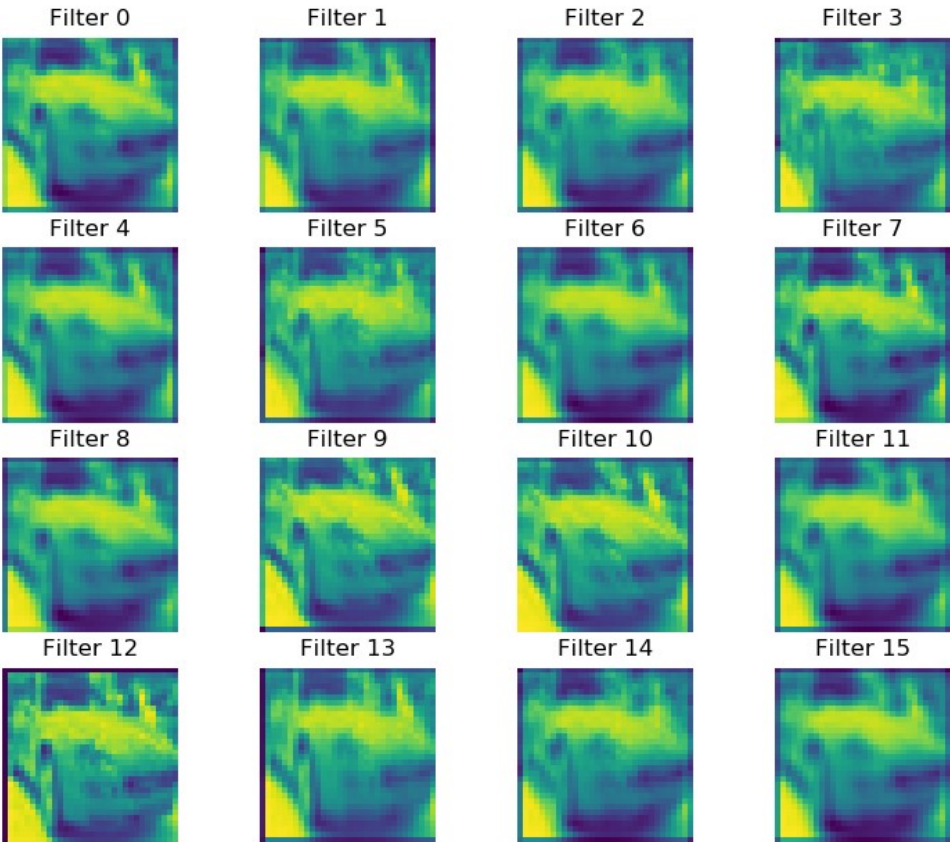


Fig. 4.4: Output after Relu

(continued from previous page)

```

4 import numpy as np
5 import matplotlib.pyplot as plt
6 import tensorflow as tf
7
8 def unpickle(file):
9     fo = open(file, 'rb')
10    dict = pickle.load(fo, encoding='latin1')
11    fo.close()
12    return dict
13
14 def read_data(directory):
15     # get classes
16     names = unpickle('{} /batches.meta'.format(directory))['label_names']
17
18     # get data and labels
19     data, labels = [], []
20     for i in range(1, 6): # there are 5 files: data_batch_1 to data_batch_5
21         filename = '{} /data_batch_{}'.format(directory, i)
22         batch_data = unpickle(filename)
23         # dict_keys(['batch_label', 'labels', 'data', 'filenames'])
24         # print(batch_data.keys())
25         if len(data) == 0:
26             data = batch_data['data'] # extract data from the dict
27             labels = batch_data['labels'] # extract key from the dict
28         else:
29             # append data from next dict
30             data = np.vstack((data, batch_data['data']))
31             # append labels from next dict
32             labels = np.hstack((labels, batch_data['labels']))
33     return names, data, labels
34
35
36 def show_conv_results(data, rows=4, cols=4):
37     plt.figure()
38     rows, cols = rows, cols
39     for i in range(np.shape(data)[3]):
40         img = data[0, :, :, i]
41         plt.subplot(rows, cols, i + 1)
42         plt.imshow(img)
43         plt.title("Filter {}".format(i))
44         plt.axis('off')
45
46
47 if __name__ == "__main__":
48     directory = 'C:/meher/Git/cifar-10-batches-py'
49     # names = unpickle('{} /batches.meta'.format(directory))
50     # print(names['label_names']) # print class names
51     names, data, labels = read_data(directory)
52
53     item = 5
54     # print("Names: ", names)
55     # print("data: ", data[item])
56     # print("item: {}, labels: {}, name: {}".format(item,
57     #         labels[item], names[labels[item]]))
58
59     # rows, cols = 3, 3 # show 9 images
60     # for item in range(0, rows*cols):
61     #     img = np.reshape(data[item], (3, 32, 32)).transpose(1,2,0)
62     #     # img = data[item].reshape((3, 32, 32)).transpose(1,2,0)
63     #     plt.subplot(rows, cols, item+1)
64     #     plt.imshow(img)

```

(continues on next page)

(continued from previous page)

```

65     # plt.title(names[labels[item]])
66
67
68     img = data[item, :]
69     img = tf.cast(img, tf.float32)
70     img = tf.reshape(img, shape=[-1, 32, 32, 1])
71
72
73     total_conv_filt = 16
74     conv_filt_size = 3
75     b = tf.Variable(tf.random_normal([total_conv_filt]))
76     conv_filt = tf.Variable(tf.random_normal([conv_filt_size, conv_filt_size, 1, total_conv_filt]))
77     conv_with_b = tf.nn.bias_add(conv_filt, b)
78     conv_out = tf.nn.relu(conv_with_b)
79
80     k = 3
81     maxpool = tf.nn.max_pool(conv_out,
82                             ksize=[1, k, k, 1],
83                             strides=[1, k, k, 1],
84                             padding='SAME')
85
86     conv = tf.nn.conv2d(img, maxpool, strides=[1, 1, 1, 1], padding='SAME')
87
88     with tf.Session() as sess:
89         sess.run(tf.global_variables_initializer())
90         conv_val = sess.run(conv)
91         show_conv_results(conv_val, rows=4, cols=4)
92         print(np.shape(conv_val))
93
94     # plt.tight_layout()
95     plt.show()

```

- Below is the output of maxpool layer.

4.5 Implement convolution network

In the previous section, we saw the output of different layers individually. In this section, we will create a convolution network and see it's performance on CIFAR dataset.

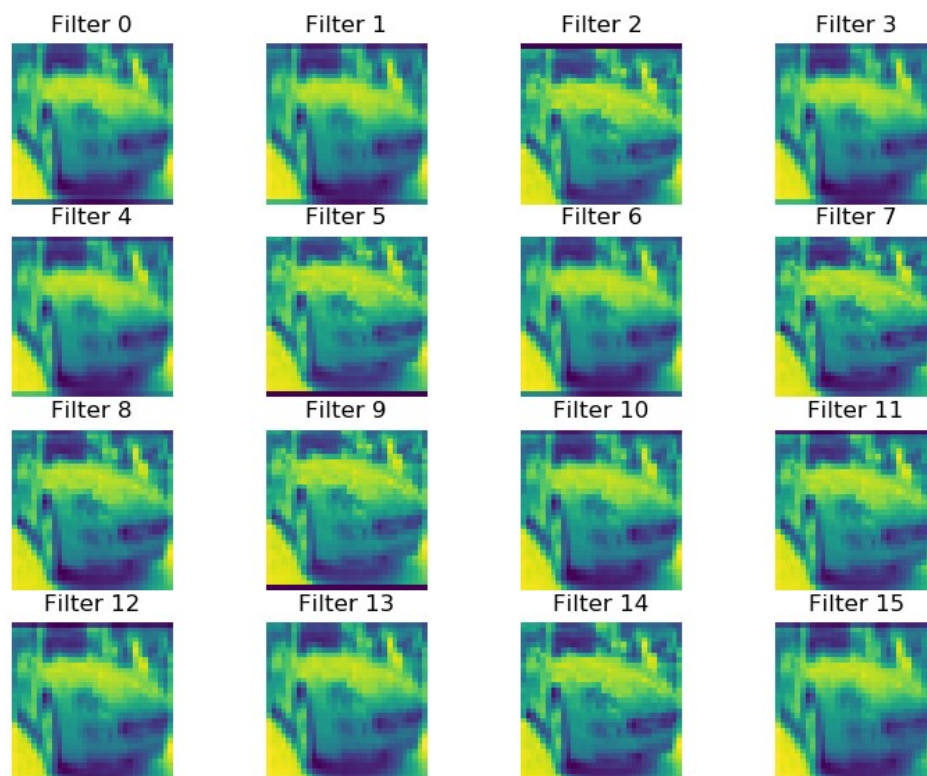


Fig. 4.5: Output of maxpool layer

Chapter 5

MNIST example

5.1 Introduction

The MNIST database (Modified National Institute of Standards and Technology database) is a dataset of hand-written digits which can be used for training various image processing systems.

In this chapter we will use MNIST data to understand the basic flow of the tensorflow coding. More specifically, we will use the Gradient-optimizer in the tensorflow graph to predict the hand-written digits.

5.2 Read and Plot numbers from “mnist” dataset

Listing 5.1: Plots the test data from mnist datasets

```
1  # mnist_ex.py
2
3  # MNIST dataset is a collection of hand-written digits-images.
4  # Training set = 55000
5  # Test set = 10000
6  # Validation set = 5000
7
8  import numpy as np
9  import tensorflow as tf
10
11 from tensorflow.examples.tutorials.mnist import input_data
12
13 from helper_functions import plot_mnist_image
14
15
16 # data will be downloaded to './data/MNIST'
17 # one_hot convert each number to vector with single 1,
18 # e.g. 0 = [1, 0, 0, 0, 0, 0, 0, 0] (assuming total 8 labels)
19 # and 4 = [0, 0, 0, 0, 1, 0, 0, 0]
20
21 # # without oneshot labels
22 # data = input_data.read_data_sets("data/MNIST/")
23 # print("label for first 5 elements:\n {}".format(data.test.labels[0:5]))
24 # # label for first 5 elements:
25 # #  [7 2 1 0 4]
26
27 # with oneshot labels
28 data = input_data.read_data_sets("data/MNIST/", one_hot=True)
29 print("label for first 5 elements:\n {}".format(data.test.labels[0:5, :]))
```

(continues on next page)

(continued from previous page)

```

30 # label for first 5 elements:
31 # [[0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
32 # [0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
33 # [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
34 # [1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
35 # [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]]
36
37
38 # print("Size of: ")
39 # print("Traing set: {}".format(len(data.train.labels))) # 55000
40 # print("Test set: {}".format(len(data.test.labels))) # 10000
41 # print("Validation set: {}".format(len(data.validation.labels))) # 5000
42
43
44 # MNIST image shape is 28x28 pixel
45 img_size = 28
46 img_shape = (img_size, img_size) # shape of image
47 img_flat_size = img_size * img_size # size after flattting the matrix
48 num_labels = 10 # i.e. 0-9
49
50 # convert oneshot to number for displaying in plots
51 data.test.numbers = np.array([label.argmax() for label in data.test.labels])
52
53 # plot first 9 images
54 images = data.test.images[0:9]
55 true_images = data.test.numbers[0:9]
56 # plot images
57 plot_images = plot_mnist_image(images=images,
58     label_true=true_images,
59     label_predict=None, # as no prediction made yet
60     img_shape=img_shape
61 )

```

Listing 5.2: Helper function to plot the mnist data

```

1 # helper_functions.py
2
3 import matplotlib.pyplot as plt
4
5 def plot_mnist_image(images, label_true, label_predict, img_shape=(28, 28)):
6     fig, axes = plt.subplots(3, 3) # 3x3 plots
7
8     for i, ax in enumerate(axes.flat):
9         # plot image
10         ax.imshow(images[i].reshape(img_shape), cmap='binary')
11
12         # show true and predicted labels
13         if label_predict is None:
14             xlabel = "True: {}".format(label_true[i])
15         else:
16             xlabel = "True: {}, Predict: {}".format(label_true[i], label_predict[i])
17
18         ax.set_xlabel(xlabel)
19
20 plt.show()

```

5.3 Create tensorflow graph

- create placeholder for input

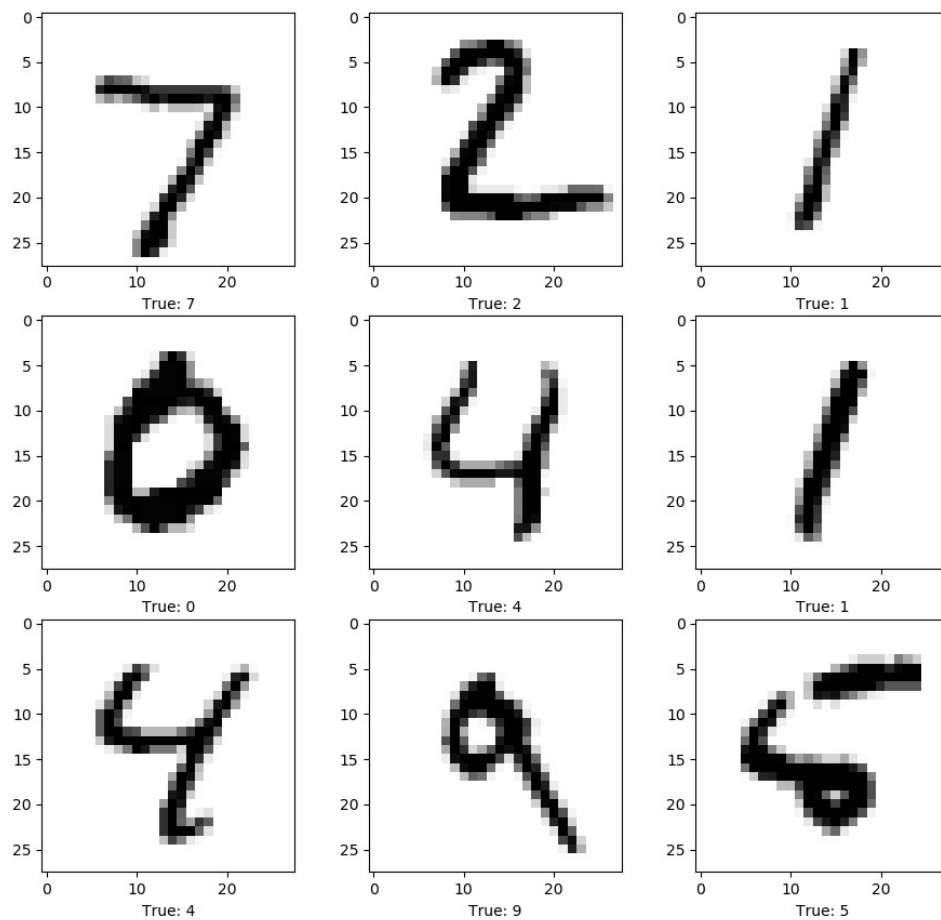


Fig. 5.1: First nine numbers in test data

Listing 5.3: create placeholder for input

```

1 # mnist_ex.py
2
3 [...]
4
5 # create placeholder for input value
6 # type float32, rows size = any number, column size = img_flat_size
7 x = tf.placeholder(tf.float32, [None, img_flat_size]) # store data of image
8 y = tf.placeholder(tf.float32, [None, num_labels])    # store one_hot labels
9 # store numbered labels i.e. corresponding values to one_hot labels
10 y_number = tf.placeholder(tf.int64, [None])

```

- create variable of tensorflow

Listing 5.4: create variables of tensorflow

```

1 # mnist_ex.py
2
3 [...]
4
5
6 # create variables of tensorflow
7 # weights
8 weights = tf.Variable(tf.zeros([img_flat_size, num_labels]))
9 # bias
10 bias = tf.Variable(tf.zeros([num_labels]))

```

5.4 Create model

A simple neural network model can be defined as “**output = input * weights + bias**”. This model can be defined using tensorflow as shown in [Listing 5.5](#)

Listing 5.5: create model

```

1 # mnist_ex.py
2
3 [...]
4
5 # create model
6 logits = tf.matmul(x, weights) + bias
7
8 # convert above results into probability density using
9 # softmax fuction i.e. sum of the vector will be 1
10 y_pred = tf.nn.softmax(logits)
11 # store the index-number which has highest value
12 y_pred_number = tf.argmax(y_pred, dimension=1)

```

Note: The [Listing 5.5](#) only creates the model and does not execute the statement.

5.5 Execute the model

Let’s execute the model now without any optimization. Since, all the weights are initialized as zero, and no optimization is performed, therefore all the numbers will be predicted as zero.

Listing 5.6: Execute the model

```

1  # mnist_ex.py
2
3  [...]
4
5
6  # Create session and check the result:
7  # since all the weights are zero, # therefore all the values
8  # will be detected as zero
9
10 # create session
11 session = tf.Session()
12 # initialize variables
13 session.run(tf.global_variables_initializer())
14
15 # assign value to placeholder
16 feed_dict = {
17     x: data.train.images,
18     y: data.train.labels
19 }
20
21 # calculate y_pred_number and save in 'result'
22 result = session.run(y_pred_number, feed_dict=feed_dict)
23 # plot first 9 result
24 plot_images = plot_mnist_image(images=images,
25     label_true=true_images,
26     label_predict=result[0:9],
27     img_shape=img_shape
28 )
29
30 # Sum is 0 as all the prediction are zero
31 print(sum(result)) # 0

```

- The completed code is shown below,

Listing 5.7: Complete 'mnist_ex.py' code till now

```

1  # mnist_ex.py
2
3  # MNIST dataset is a collection of hand-written digits-images.
4  # Training set = 55000
5  # Test set = 10000
6  # Validation set = 5000
7
8  import numpy as np
9  import tensorflow as tf
10
11 from tensorflow.examples.tutorials.mnist import input_data
12
13 from helper_functions import plot_mnist_image
14
15
16 # data will be downloaded to './data/MNIST'
17 # one_hot convert each number to vector with single 1,
18 # e.g. 0 = [1, 0, 0, 0, 0, 0, 0, 0] (assuming total 8 labels)
19 # and 4 = [0, 0, 0, 0, 1, 0, 0, 0]
20
21 # # without oneshot labels
22 # data = input_data.read_data_sets("data/MNIST/")
23 # print("label for first 5 elements:\n {}".format(data.test.labels[0:5]))
24 # # label for first 5 elements:

```

(continues on next page)

(continued from previous page)

```

25 # # [7 2 1 0 4]
26
27 # with oneshot labels
28 data = input_data.read_data_sets("data/MNIST/", one_hot=True)
29 print("label for first 5 elements:\n {}".format(data.test.labels[0:5, :]))
30 # label for first 5 elements:
31 # [[0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
32 # [0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
33 # [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
34 # [1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
35 # [0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]]
36
37
38 # print("Size of: ")
39 # print("Traing set: {}".format(len(data.train.labels))) # 55000
40 # print("Test set: {}".format(len(data.test.labels))) # 10000
41 # print("Validation set: {}".format(len(data.validation.labels))) # 5000
42
43
44 # MNIST image shape is 28x28 pixel
45 img_size = 28
46 img_shape = (img_size, img_size) # shape of image
47 img_flat_size = img_size * img_size # size after flattening the matrix
48 num_labels = 10 # i.e. 0-9
49
50 # convert oneshot to number for displaying in plots
51 data.test.numbers = np.array([label.argmax() for label in data.test.labels])
52
53 # plot first 9 images
54 images = data.test.images[0:9]
55 true_images = data.test.numbers[0:9]
56 # plot images
57 plot_images = plot_mnist_image(images=images,
58                                label_true=true_images,
59                                label_predict=None, # as no prediction made yet
60                                img_shape=img_shape
61 )
62
63
64 # create placeholder for input value
65 # type float32, rows size = any number, column size = img_flat_size
66 x = tf.placeholder(tf.float32, [None, img_flat_size]) # store data of image
67 y = tf.placeholder(tf.float32, [None, num_labels]) # store one_hot labels
68 # store numbered labels i.e. corresponding values to one_hot labels
69 y_number = tf.placeholder(tf.int64, [None])
70
71
72 # create variables of tensorflow
73 # weights
74 weights = tf.Variable(tf.zeros([img_flat_size, num_labels]))
75 # bias
76 bias = tf.Variable(tf.zeros([num_labels]))
77
78
79 # create model
80 logits = tf.matmul(x, weights) + bias
81
82 # convert above results into probability density using
83 # softmax fuction i.e. sum of the vector will be 1
84 y_pred = tf.nn.softmax(logits)
85 # store the index-number which has highest value

```

(continues on next page)

(continued from previous page)

```

86 y_pred_number = tf.argmax(y_pred, dimension=1)
87
88
89 # Create session and check the result:
90 # since all the weights are zero, # therefore all the values
91 # will be detected as zero
92
93 # create session
94 session = tf.Session()
95 # initialize variables
96 session.run(tf.global_variables_initializer())
97
98 # assign value to placeholder
99 feed_dict = {
100     x: data.train.images,
101     y: data.train.labels
102 }
103
104 # calculate y_pred_number and save in 'result'
105 result = session.run(y_pred_number, feed_dict=feed_dict)
106 # plot first 9 result
107 plot_images = plot_mnist_image(images=images,
108     label_true=true_images,
109     label_predict=result[0:9],
110     img_shape=img_shape
111 )
112
113 # Sum is 0 as all the prediction are zero
114 print(sum(result)) # 0

```

5.6 Use Gradient optimizer for prediction

We need to define the ‘cost function’ to use the Gradient optimizer. The optimizer will try to reduce the cost function. In the below code, ‘cross entropy’ is used as the cost function.

Listing 5.8: Use Gradient optimizer for prediction

```

1 # mnist.py
2
3 # MNIST dataset is a collection of hand-written digits-images.
4 # Training set = 55000
5 # Test set = 10000
6 # Validation set = 5000
7
8 import numpy as np
9 import tensorflow as tf
10
11 from tensorflow.examples.tutorials.mnist import input_data
12
13 from helper_functions import plot_mnist_image
14
15
16 # data will be downloaded to './data/MNIST'
17 # one_hot convert each number to vector with single 1,
18 # e.g. 0 = [1, 0, 0, 0, 0, 0, 0, 0] (assuming total 8 labels)
19 # and 4 = [0, 0, 0, 0, 1, 0, 0, 0]
20
21 # # without oneshot labels

```

(continues on next page)

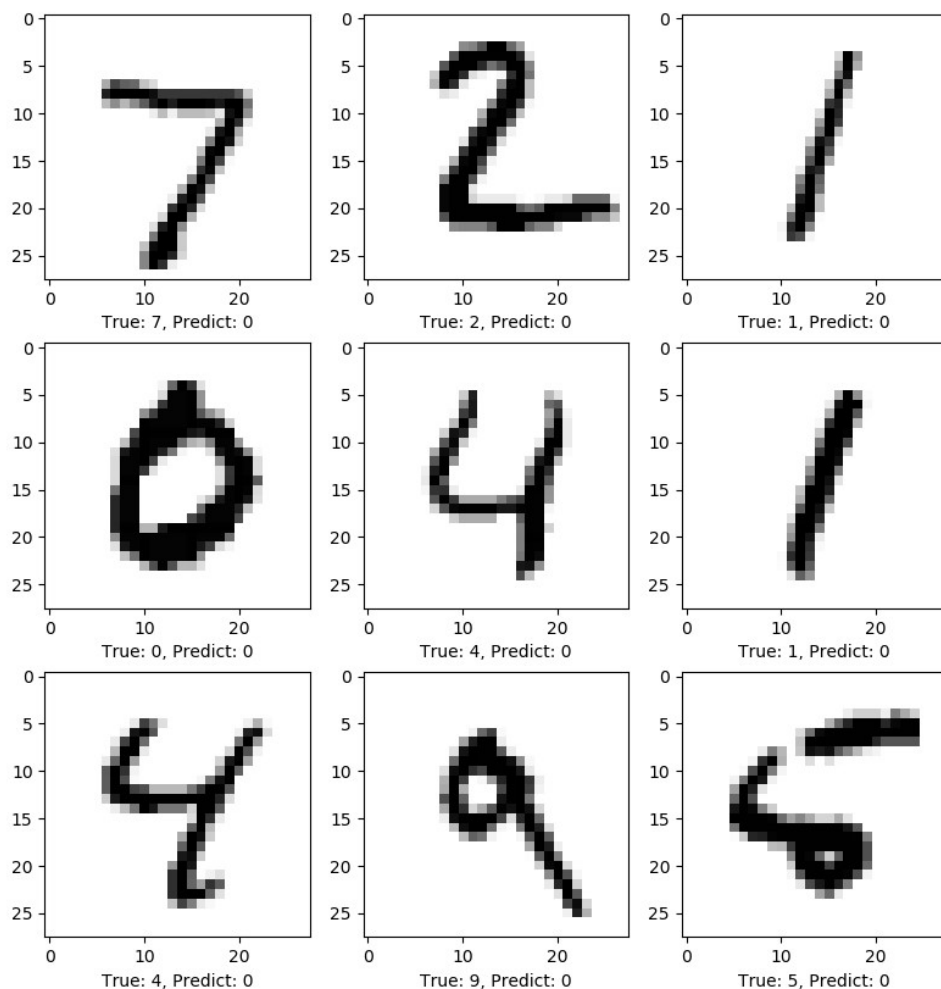


Fig. 5.2: All the numbers are predicted as 0

(continued from previous page)

```

22 # data = input_data.read_data_sets("data/MNIST/")
23 # print("label for first 5 elements:\n {}".format(data.test.labels[0:5]))
24 # # label for first 5 elements:
25 # # [7 2 1 0 4]
26
27 # with oneshot labels
28 data = input_data.read_data_sets("data/MNIST/", one_hot=True)
29 # print("label for first 5 elements:\n {}".format(data.test.labels[0:5, :]))
30 # # label for first 5 elements:
31 # # [[0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
32 # # [0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
33 # # [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
34 # # [1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
35 # # [0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]]
36
37
38 # print("Size of: ")
39 # print("Training set: {}".format(len(data.train.labels))) # 55000
40 # print("Test set: {}".format(len(data.test.labels))) # 10000
41 # print("Validation set: {}".format(len(data.validation.labels))) # 5000
42
43
44 # MNIST image shape is 28x28 pixel
45 img_size = 28
46 img_shape = (img_size, img_size) # shape of image
47 img_flat_size = img_size * img_size # size after flattening the matrix
48 num_labels = 10 # i.e. 0-9
49
50 # convert oneshot to number for displaying in plots
51 data.test.numbers = np.array([label.argmax() for label in data.test.labels])
52
53 # plot first 9 images
54 images = data.test.images[0:9]
55 true_images = data.test.numbers[0:9]
56 # # plot images
57 # plot_images = plot_mnist_image(images=images,
58 # label_true=true_images,
59 # label_predict=None, # as no prediction made yet
60 # img_shape=img_shape
61 # )
62
63
64 # create placeholder for input value
65 # type float32, rows size = any number, column size = img_flat_size
66 x = tf.placeholder(tf.float32, [None, img_flat_size]) # store data of image
67 y = tf.placeholder(tf.float32, [None, num_labels]) # store one_hot labels
68 # store numbered labels i.e. corresponding values to one_hot labels
69 y_number = tf.placeholder(tf.int64, [None])
70
71
72 # create variables of tensorflow
73 # weights
74 weights = tf.Variable(tf.zeros([img_flat_size, num_labels]))
75 # bias
76 bias = tf.Variable(tf.zeros([num_labels]))
77
78
79 # create model
80 logits = tf.matmul(x, weights) + bias
81
82 # convert above results into probability density using

```

(continues on next page)

(continued from previous page)

```

83 # softmax fuction i.e. sum of the vector will be 1
84 y_pred = tf.nn.softmax(logits)
85 # store the index-number which has highest value
86 y_pred_number = tf.argmax(y_pred, dimension=1)
87
88
89 # Define cost-function for optimization : i.e. corss-entropy here
90 cross_entropy = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits_v2(labels=y, logits=logits))
91 # use optimizer on cost function "cross-entropy"
92 optimizer = tf.train.GradientDescentOptimizer(0.5).minimize(cross_entropy)
93
94 # Evaluate accuracy
95 # correct prediction i.e. predicted = actual
96 correct_prediction = tf.equal(y_pred_number, y_number)
97 # accuracy = correct_prediction/total_images
98 accuracy = tf.reduce_mean(tf.cast(correct_prediction, tf.float32))
99
100
101 # Create session and check the result:
102 # since all the weights are zero, # therefore all the values
103 # will be detected as zero
104
105 # create session
106 session = tf.Session()
107 # initialize variables
108 session.run(tf.global_variables_initializer())
109
110 # assign value to placeholder
111 feed_dict = {
112     x: data.train.images,
113     y: data.train.labels
114 }
115
116
117 session.run(optimizer, feed_dict=feed_dict)
118 # # or run in batch (more accurate than above)
119 # repeat = 4000
120 # for i in range(repeat): # run 'repeat' times by selecting 100 data randomly
121 #     # load data in a batch of 100
122 #     batch = data.train.next_batch(100)
123 #     # run optimizer for 100 data
124 #     session.run(optimizer, feed_dict={x: batch[0], y: batch[1]})
125
126 feed_dict_accuracy = {
127     x: data.test.images,
128     y: data.test.labels,
129     y_number: data.test.numbers
130 }
131
132 prediction, accuracy = (session.run([y_pred_number, accuracy], feed_dict=feed_dict_accuracy))
133 print("Accuracy = ", accuracy) # Accuracy = 0.6705, for batch method->Accuracy = 0.9238
134
135 # plot first 9 result
136 plot_images = plot_mnist_image(images=images,
137     label_true=true_images,
138     label_predict=prediction[0:9],
139     img_shape=img_shape
140 )

```

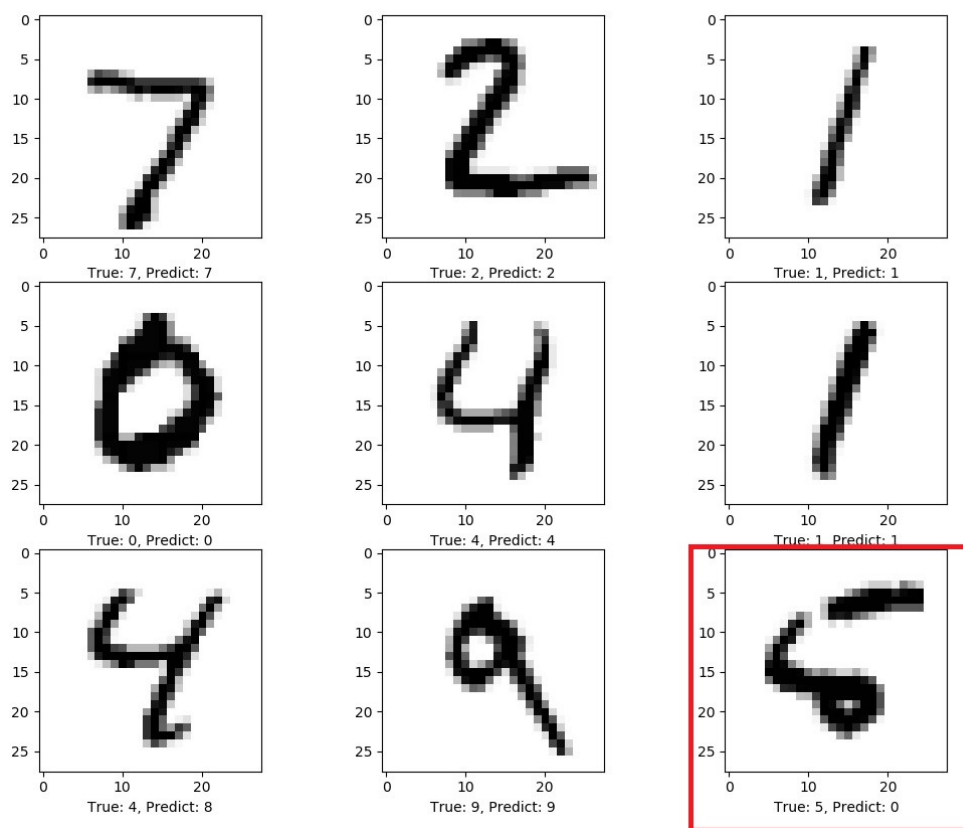


Fig. 5.3: Accuracy improved by using Gradient optimizer for prediction